Approximation and Representation

B. Grey Vandeberg

4 Grey's Graphs

So, while studying the material of the previous chapter, I have come across something which I can only describe as, well, scary. Let me tell you the tale in full, starting from the beginning as I recall it.

The Gumbel Connection

The reader of the previous chapter's exercises has, at this point, been belabored with the **standard Gumbel distribution** with pdf

$$f_G(t) = e^{-e^{-t}}, \qquad t \in \mathbb{R}.$$

The standard Gumbel, of course, may be generalized to a more general location-scale family:

Definition 4.0.1 (Gumbel Location-Scale)

We say $G \sim \text{Gumbel}(\mu, \beta)$ if the density of $G \in \mathcal{G}$ is

$$f_G(x \mid \mu, \beta) := \frac{1}{\beta} e^{-\left(\frac{x-\mu}{\beta}\right)} \cdot e^{-e^{-\left(\frac{x-\mu}{\beta}\right)}}, \qquad x \in \mathcal{G} = \mathbb{R}, \qquad \mu \in \mathbb{R}, \qquad \beta > 0,$$

where we note the Jacobian $e^{-(x-\mu)/\beta} \cdot 1/\beta > 0$ for $\mu \in \mathbb{R}$ and $\beta > 0$. Furthermore, note that the standard Gumbel is recovered by taking Gumbel($\mu = 0$, $\beta = 1$).

There was something quite interesting about the standard Gumbel's cumulants: we found that they were *exactly*

The Main Result

Definition 4.0.2 (Graph)

A **graph** $G = (V, E)$ consists of a finite set of **vertices** $V = \{v_1, v_2, \dots, v_n\}$, together with a finite set of **edges** $E = \{e_1, e_2, \dots, e_m\}$.

Here, we consider each edge $e_k = \{v_i, v_j\}$ to consist of an *unordered* pair of vertices — i.e., we consider our graphs to be **undirected**, so that, for instance, we have $\{v_i, v_j\} = \{v_j, v_i\}$. "We don't allow duplicate edges." (In particular, this contrasts with a *directed* graph, whose edges

would take the form $(v_i, v_j) \neq (v_j, v_i)$.) Throughout, when we refer to "a graph," we *always* mean an undirected one — nevertheless, there are many instances where we will *impose* some kind of directed structure *over* an undirected graph.

One observes that the graph $G = (V, E)$ consists of n vertices and m edges — indeed, we write

$$|V| = n, \qquad |E| = m.$$

The reader of the last chapter observes that, though we *define* the edge $e_k = \{v_i, v_j\}$ relative to the vertices v_i, v_j, the edge serves as something of a *representative* of the vertices themselves — indeed, we say that the edge e_k is **incident** to v_i, v_j. We might imagine the term's utility comes down to whether we're "asking which vertices are at opposite ends of a *given* edge", or "asking which vertices are connected by an edge *at all*."

Finally, we say that the **degree** $d(v)$ of a vertex $v \in V$ is the number of edges incident to v.

We now discuss the *traversability* of graphs, as well as some facilitating mechanisms. Indeed, we want to enforce — immediately and without apology — the idea that G is not an "abstract combinatorial object," but a **blueprint**. Quite literally: place yourself in the mind of an electrician, handed a schematic of n junction boxes connected by m cables. At this point, we imagine the blueprint is *fixed* — we cannot lay cables ourselves, but we're going to scrutinize the other electricians' plans. Maybe we're the foreman.

A good foreman, handed a wiring schematic, immediately raises several concerns:

- "Is this circuit **connected**? Can I tract a route from any box to any other by following a series cables — or has the crew left some section of the building in the dark?"
- "Is this circuit **efficient**? If I can reach box v_j from box v_i by two different routes, then one of them is redundant. The crew has wasted cable on a cycle — a closed loop does nothing but cost material."
- "What is the **leanest** version of this plan that still keeps every box powered? That is — what is the *smallest subset* of cables $\{e_1, \dots, e_p\}$ I can sign off while maintaining full connectivity?"

The reader is not surprised to learn each of these questions has a formal characterization.

Definition 4.0.3 (Path)

Let $v_i, v_j \in V$ be two vertices on a graph $G = (V, E)$. We say that a **path** from v_i to v_j is a sequence of distinct vertices

$$\{v_i = u_0, u_1, u_2, \dots, u_\ell = v_j\} \subset V$$

such that each consecutive pair $\{u_i, u_j\}$ is an edge in E. (Clearly, we could also characterize this in terms of incident edges $e_k \in E$.)

We can also observe that there are necessarily $(\ell+1)-1 = \ell$ edges in any such path $\{u_0, \dots, u_\ell\}$ — in particular, we call the number ℓ of edges traversed the **length** of the path.

In the electrician's language, a path is a route through the blueprint that never revists a junction box. We walk from v_i to v_j, following cables, and we *don't double back*.

Definition 4.0.4 (Connected Graph)

A graph $G = (V, E)$ is **connected** if, for every pair of vertices $v_i, v_j \in V$, there exists a path from v_i to v_j.

This is the foreman's first concern, now made precise: the schematic is connected if and only if every junction box can be reached from every other by following paths of cables. Nothing's orphaned in a connected graph.

Definition 4.0.5 (Cycle)

A **cycle** in G is a sequence of distinct vertices $u_0, u_1, \ldots, u_\ell$ (with $\ell \geq 2$) such that $\{u_k, u_{k+1}\}$ is an edge for each $k = 0, \ldots, \ell - 1$, and furthermore, where $\{u_{\ell-1}, u_0\}$ is *also* an edge. That is: a cycle is nothing more than a *path that returns to where it started.*

This is the foreman's second concern. A cycle means redundancy — if we can get electricity from u_0 to u_3 by routing power through u_1 and u_2, it would be a waste of cable to connect u_0 and u_3 directly.

Definition 4.0.6 (Tree)

A **tree** is a *connected graph* with *no cycles.*

We might imagine the tree is what we the foremen would've drawn themselves if we'd had the chance. Every box is reachable, and there is no waaste. The tree is the *leanest* possible schematic — yank any cable and the circuit breaks, add any cable and you create a cycle. The tree is the *unique* sweet spot between "not enough" and "too much."

Theorem 4.0.7 (Trees on n Vertices)

A tree on $|V| = n$ vertices has exactly $n - 1$ edges.

Proof:

First, we observe the *connectivity* of trees trivially ensure *at least* $n - 1$ edges.

When we *additionally* consider that trees are *acyclic* graphs, we conclude any path through pairs of distinct vertices in V must consist of *at most* $n - 1$ edges.

Indeed, suppose we have a connected graph on n vertices, but with n *or more edges.* Connectivity guarantees a path between any two vertices — so, in particular, we may build a spanning tree T by greedily adding edges that connect new vertices, stopping when all n vertices are reached. This requires exactly $n - 1$ edges. But we assumed we had at least n edges in total, so at least one edge $e = \{v_i, v_j\}$ was *not* used in T. Since T is a spanning tree, it already contains a path from v_i to v_j — and the edge e, together with that path, closes a cycle. The graph is not acyclic.

Contrapositively, we see acyclicity forces $m \leq n-1$, connectivity forces $m \geq n-1$, and a tree — being both — has exactly $m = n-1$.

$\|$

Now, we consider a speicial class of trees.

Definition 4.0.8 (Spanning Tree)

Let $G = (V, E)$ be a connected graph. A **spanning tree** of G is a subgraph $T = (V, E_T)$ — i.e., with V the same set of original vertices, but with $E_T \subseteq E$ typically a strict subset of edges $|E_T| < |E|$ — such that:

(i) T contains *every* vertex of G (it *spans* V), and

(ii) T is a tree — i.e., T is connected and acyclic.

By our theorem on trees, any spanning tree of G uses exactly $|V| - 1$ edges. In particular, if G has $|E| = m$ edges and $|V| = n$ vertices, then we may deduce that the set E_T of edges of a spanning tree T *discards* exactly

$$L := m - (n-1) = m - n + 1$$

edges from the original set E. We define this number quite deliberately:

Definition 4.0.9 (Loop Number)

If $G = (V, E)$ is a graph with $|V| = n$ vertices and $|E| = m$ edges, and if $T = (V, E_T)$ with $|E_T| = n - 1$ is a spanning tree of G, then we define the **loop number** of G to be

$$L := m - n + 1.$$

We observe that the loop number L of a graph G counts the number of "redundant cables" e in the original schematic E — the ones the forman would remove, were they to use "tree-ness" as their criterion for "blueprint suitability."

But let us dwell on the loop number $L = m-n+1$ for a moment, because it is doing something rather pretty. If G is *already* a tree, then $m = n - 1$, and we have $L = (n-1) - n + 1 = 0$ — indeed, there was *nothing* for the foreman to cut. Supposing G has *one extra cable* beyond what a tree requires, then $L = 1$: exactly one cycle, one redundancy, one edge the foreman could pull from the circuit and still power the graph.

So: the foreman has a graph $G = (V, E)$ with n vertices, m edges, and loop number $L = m - n + 1$. The foreman knows that any spanning tree T uses exactly $n - 1$ edges from E, discarding the remaining L. But here is the foreman's next question — a question which, until now, may not have seemed important:

How many different spanning trees does this graph have?

The foreman is, after all, a practical person. If there are *many* spanning trees, the foreman has *options* — many ways to strip the blueprint down to its leanest form, and the foreman (being lazy, as all good foremen are) will naturally want the *cheapest* one. If there are *few* spanning trees, the foreman is cornered: the blueprint is tight, the wiring plan is nearly forced, and there is little room to optimize.

This is not an idle distinction.

Definition 4.0.10 (Number of Spanning Trees)

Given a graph $G = (V, E)$, we denote the number of spanning trees of G by $N_T(G)$.

For instance, a cycle graph C_n on n vertices — i.e., a single closed loop of n boxes connected by n cables — has exactly n spanning trees: remove any one of the n edges and you get a path which is forced to be a tree. On the other hand, a path graph P_n on n vertices leads directly from one vertex to another with no loops — in this case, P_n has *exactly 1* spanning tree: itself.

Indeed, a graph that is "barely connected" — held together by a single or couple bottleneck cables — has few spanning trees, because every spanning tree must include that bottleneck. A graph that is "richly connected" — with many alternate routes between every pair of vertices — has many spanning trees, because the foreman has many ways to strip the blueprint down.

Computing $N_T(G)$ is quite desirable then, precisely because it gives a "one-number summary" relaying something about "how connected the graph is." The foreman might want to use $N_T(G)$ to recognize graphs which are immediately serviceable, and which are most susceptible to improvements. One of the most celebrated formulas — due to Cayley — counts $N_T(K_n)$, the number of spanning trees on the **complete graph** on n vertices K_n (in which every pair of vertices is connected by an edge.) But to reach Cayley's formula, we need a more general means for computing $N_T(G)$: that tool is the **matrix-tree theorem**, which we state in just a tick.

Before we do, we need to introduce one further generalization of our setup: thus far, the foreman has treated all edges (cables) as *interchangeable* — the only question was "which cables to keep" and "which to pull." But we often want to imagine that the cables come in different flavors: some are long runs of expensive copper, others short jumps of cheap aluminum. The foreman, being lazy *and* frugal, wants to minimize total cost — and for that, we need to attach a **cost** to each cable.

Definition 4.0.11 (Weighted Graph)

A **weighted graph** $(G, \mathbf{x})$ is a graph $G = (V, E)$ together with a vector of **edge weights** $\mathbf{x} = (x_1, x_2, \ldots, x_m) \in \mathbb{R}^m_{>0}$, where $x_k > 0$ is the weight assigned to edge $e_k \in E$.

Just as the ordinary and exponential generating functions were *formal power series*, we consider the weights $x_k \in \mathbf{x}$ associated with a weighted graph $(G, \mathbf{x})$ to be *formal variables* — placeholders that allow us to track how the graph's combinatorial structure *interacts* with the weights algebraically.

Remark.

Relating to our running metaphor, we might imagine that the foreman assigns the weight x_k to each cable $e_k = \{v_i, v_j\}$ in the blueprint according to *how much it costs* to lay that particular cable. But what does the foreman's money actually *buy*? A more expensive cable is not simply a "more expensive cable" — it is a *better* cable. Thicker copper, shorter run, lower resistance — but at the end of the day, all of these just mean we "paid more for something that conducts electricity better." The foreman who spends x_k on a cable e_k between buildings v_i and v_j is *spending their money* on **conductance**: the equivalence one should fix in their mind is, "the foreman spends x_k, and that x_k is spent directly on the edge e_k's conductance, so we consider x_k to *be* the conductance."

In the formal language of circuits, **conductance** is the rate at which electricity flows through a cable per unit of "push" applied across it. A cable with conductance $x_k = 10$ carries ten times the current of a cable with conductance $x_k = 1$ under the same voltage difference. A cable with $x_k = 0$ carries no current at all — a severed wire, an edge that might as well not exist.

For now, though, we treat the weights as formal variables — we are not yet "turning on" the network. We are still in the regime of combinatorics: tallying spanning trees, forming polynomials, counting options.

We proceed onward: given a spanning tree T of G, the tree uses $n-1$ edges and discards the remaining $L = m - (n-1)$. We can form two natural monomials from any such tree:

- The product of the weights of edges *in* the tree, giving the $(n-1)$ degree polynomial
$$\prod_{e_k \in T} x_k \in \Pi_{n-1}.$$
(Here, we use Π_k to denote the general class of polynomials of degree at most k — this is a slight change from the last chapter, where the notation denoted *monic* polynomials of degree at most k. The distinction here is largely unimportant.)
- The products of the weights of edges *not in* the tree, giving the $L = (m-n+1)$-degree polynomial
$$\prod_{e_k \notin T} x_k \in \underbrace{\Pi_{m-n+1}}_{\equiv \Pi_L}$$

Thus, if we imagine fixing a particular tree $T = (V, E_T)$ on a graph $G = (V, E)$, we imagine that $E = E_T \cup E_{T^C}$ — "the original edges E of G decompose as a set of edges in the tree E_T, and a set of edges not in the tree E_T^C" — relative to which we form the two polynomials above. These polynomials are defined for *every* spanning tree in G — all $N_T(G)$ of them. In particular, we might imagine that a quantity of interest would be the contributions of *all* the products of the forms above. Indeed, we have the following pair of related definitions.

Definition 4.0.12 (Kirchoff Polynomial)

The **Kirchoff polynomial** of a weighted graph $(G, \mathbf{x})$ is defined as

$$K_G(x_1, \dots, x_m) := \sum_{\substack{\text{spanning} \\ \text{trees } T}} \prod_{e_k \in T} x_k \ \in \ \Pi_{n-1}.$$

Observe that K_G is a polynomial of degree $n-1$ in m variables $x_1, \dots, x_m \in \mathbf{x}$. Each of its $N_T(G)$ monomials has coefficient 1, and each monomial corresponds to exactly one spanning tree.

Definition 4.0.13 (First Symanzik Polynomial)

The **first Symanzik polynomial** of a weighted graph $(G, \mathbf{x})$ is defined as

$$\psi_G(x_1, \dots, x_m) := \sum_{\substack{\text{spanning} \\ \text{trees } T}} \prod_{e_k \notin T} x_k \ \in \ \Pi_{m-n+1}.$$

ψ_G is a polynomial of degree equal to the loop number $L = m - n + 1$ in m variables. Each of its $N_T(G)$ monomials has coefficient 1, and each monomial corresponds to exactly one spanning tree.

(Though this is officially the *first* Symanzik polynomial, we will often refer to it as *the* Symanzik polynomial; otherwise, we will distinguish explicitly as the need arises.)

Let us see these polynomials in action on the simplest possible example that is not itself a tree: the **triangle graph** K_3. Indeed, define $K_3 = (V, E)$, with

$$V = \{v_1, v_2, v_3\}, \qquad E = \{e_1, e_2, e_3\},$$

and in particular, where

$$e_1 = \{v_1, v_2\}, \quad e_2 = \{v_1, v_3\}, \quad e_3 = \{v_2, v_3\}.$$

We assign weights $\mathbf{x} = (x_1, x_2, x_3)$ to the three edges (e_1, e_2, e_3) respectively.

Here, we have $|V| = n = 3$ vertices and $|E| = m = 3$ edges, so

$$L = m - n + 1 = 1;$$

in particular, we intuit $L = 1$ to mean that, viewed as the foreman's blueprint schematic, the triangle graph has *exactly one redundant cable*.

So, any spanning tree T of K_3 uses exactly $n - 1 = 2$ edges, and discards the remaining $L = m - (n-1) = 1$. Since the triangle has only three edges, discarding any one of the edges still yields a path through the graph — we can deduce that there are $N_T(K_3) = 3$ spanning trees through K_3, given by:

Tree	Edges kept	Edge discarded	Description
T_1	$\{e_1, e_2\}$	e_3	v_1 connects to both v_2 and v_3; the edge v_2–v_3 is removed
T_2	$\{e_1, e_3\}$	e_2	v_2 connects to both v_1 and v_3; the edge v_1–v_3 is removed
T_3	$\{e_2, e_3\}$	e_1	v_3 connects to both v_1 and v_2; the edge v_1–v_2 is removed

Now, to compute the Kirchoff and Symanzik polynomials. The Kirchoff polynomial tallies each tree by the edges it *keeps*. We see that T_1 *keeps* the edges e_1 and e_2, whose corresponding weights are x_1 and x_2 — in particular, the contribution of T_1 to $K_{K_3}(\mathbf{x})$ is

$$\sum_{e_k \in T_1} x_k = x_1 x_2.$$

The other trees are entirely symmetric, and we deduce that

$$K_{K_3}(x_1, x_2, x_3) = \sum_{\substack{\text{spanning} \\ \text{trees } T}} \prod_{e_k \in T} x_k = \underbrace{x_1 x_2}_{\text{from } T_1} + \underbrace{x_1 x_3}_{\text{from } T_2} + \underbrace{x_2 x_3}_{\text{from } T_3} .$$

We observe that the Kirchoff polynomial K_{K_3} is of degree $n - 1 = 2$ (the cross-terms count as degree-2 terms.)

The Symnanzik polynomial conducts essentially the same argument but in reverse. We see that T_1 *discards* the edge e_3 with weight x_3, and the contribution of T_1 to $\psi_{K_3}(\mathbf{x})$ is

$$\sum_{e_k \notin T_1} x_k = x_3.$$

The other trees are entirely symmetric, and we deduce that

$$\psi_{K_3}(x_1, x_2, x_3) = \sum_{\substack{\text{spanning} \\ \text{trees } T}} \prod_{e_k \notin T} x_k = \underbrace{x_3}_{\text{from } T_1} + \underbrace{x_2}_{\text{from } T_2} + \underbrace{x_1}_{\text{from } T_3}$$

We observe that the Symanzik polynomial ψ_{K_3} is of degree $L = m - (n - 1) = 1$.

We observe that each polynomial K_{K_3} and ψ_{K_3} has exactly $N_T(K_3) = 3$ monomials, one per spanning tree, each with coefficient one.

Furthermore, we observe that the Symanzik polynomial of the triangle K_3 is simply $x_1+x_2+x_3$. The "foreman's census of discarded edges" is, in this case, nothing more than the sum of the edges. It is intuitively clear that this is a special feature of $L = 1$ graphs: when each spanning tree discards *exactly one edge*, the Symanzik polynomial is a *sum of individual variables*, with *no products/cross-terms*.

The astute reader may have observed that the Kirchoff and Symanzik polynomials carry **dual** information: the foreman who "knows which edges they kept in the blueprint" knows the same information as the foreman who "knows which edges they threw away." In particular, one might imagine that if we *know* K_{K_3}, for example, we might be able to deduce ψ_{K_3} immediately without consideration of the definition above. And indeed, this happens to be the case:

Theorem 4.0.14 (Kirchoff-Symanzik Duality)

Let $(G, \mathbf{x})$ be a weighted graph with m edges. Denote by

$$X \coloneqq \prod_{k=1}^{m}$$

the product of *all* the edge weights $x_k \in \mathbf{x}$.

Then, we have that

$$\psi_G(x_1, \dots, x_m) = X \cdot K_G\left(\frac{1}{x_1}, \dots, \frac{1}{x_m}\right).$$

Proof:

Fix a spanning tree T. The corresponding monomial in the Kirchoff polynomial is $\prod_{e_k \in T} x_k$. Under the reciprocal substitution

$$x_k \mapsto 1/x_k,$$

this becomes

$$\prod_{e_k \in T} x_k \mapsto \prod_{e_k \in T} 1/x_k.$$

Multiplying by $X = \prod_{k=1}^{m} x_k$, we obtain

$$X \cdot \prod_{e_k \in T} \frac{1}{x_k} = \left(\prod_{k=1}^{m} x_k \right) \left(\prod_{e_k \in T} x_k^{-1} \right) = \prod_{e_k \notin T} x_k,$$

simply by associativity of multiplication. In particular, the factors x_k for $e_k \in T$ cancel between the competing factors, leaving precisely those factors x_k for the $L = m - n + 1$ edges *not* in the spanning tree T. But, of course, this is exactly the monomial contributed by T to the Symanzik polynomial.

Summing over all spanning trees on both sides:

$$X \cdot K_G \left(\frac{1}{x_1}, \ldots, \frac{1}{x_m} \right) = X \cdot \sum_T \prod_{e_k \in T} \frac{1}{x_k} = \sum_T \prod_{e_k \notin T} x_k = \psi_G(x_1, \ldots, x_m).$$

$\|$

We verify this for the triangle K_3: we have $X = x_1 x_2 x_3$, and

$$\begin{aligned} X \cdot K_{K_3} \left(\frac{1}{x_1}, \frac{1}{x_2}, \frac{1}{x_3} \right) &= x_1 x_2 x_3 \cdot \left(\frac{1}{x_1 x_2} + \frac{1}{x_1 x_3} + \frac{1}{x_2 x_3} \right) \\ &= x_3 + x_2 + x_1 \\ &= \psi_{K_3}(x_1, x_2, x_3). \qquad (\checkmark) \end{aligned}$$

In the foreman's language, the Kirchoff and Symanzik polynomials are *two tallies* of the *same census*. The Kirchoff polynomial records which edges from G each spanning tree *keeps*; the Symanzik polynomial records what each tree *throws away*. The duality simply says: if you "flip the perspective from edges kept to edges discards," the census doesn't change — only the bookkeeping does.

Now, the issue with all of this is that the counting gets complicated fast. Indeed, realize that for the triangle, computing the polynomials K_{K_3} and ψ_{K_3} required *enumerating* spanning trees: listing them one by one, forming the monomial for each, and summing. When there are only three edges and vertices, this is quite simple — three trees, three monomials, done. But we will see that K_4 the complete graph on four vertices, there are $N_T(K_4) = 16$ spanning trees; by the time we reach K_10 the complete graph on 10 vertices, the number of spanning trees is a whopping $N_T(K_{10}) = 10^8$.

Enumeration *does not scale*! We need a machine that computes K_G and/or ψ_G *without* listing spanning trees — and, ideally, one that runs in time proportional to the *size of the graph*, and not the *number of its spanning trees*.

That machine is given by the **weighted Laplacian matrix**.

Definition 4.0.15 (Weighted Laplacian)

Let $(G, \mathbf{x})$ be a weighted graph on n vertices $V = \{v_1, \dots, v_n\}$ with m edges. The **weighted Laplacian** $\mathbf{L}_G \in \mathbb{R}^{n \times n}$ is the matrix defined entrywise by

$$(\mathbf{L}_G)_{ij} = \begin{cases} \displaystyle\sum_{\substack{e_k \text{ incident} \\ \text{to } v_i}} x_k, & \text{if } i = j, \\ -x_k, & \text{if } e_k = \{v_i, v_j\} \in E, \\ 0, & \text{if } \{v_i, v_j\} \notin E. \end{cases}$$

In words: the diagonal entry $(\mathbf{L}_G)_{ii}$ is the total weight of edges touching vertex v_i, and the off-diagonal entries $(\mathbf{L}_G)_{ij}$ is the *negative* of the weight x_k of the edge e_k connecting v_i to v_j (or zero, if no such edge exists.)

We observe that for the class of undirected graphs we're considering currently, the weighted Laplacian $\mathbf{L}_G$ *must* be symmetric. Indeed, a great deal of utility comes out of the symmetry of $\mathbf{L}_G$.

Let us compute $\mathbf{L}_{K_3}$ for the triangle. We have three vertices $\{v_1, v_2, v_3\}$ and three edges $e_1 = \{v_1, v_2\}, e_2 = \{v_1, v_3\}, e_3 = \{v_1, v_2\}$ with associated weights (x_1, x_2, x_3). The diagonal entries are

$$(\mathbf{L}_{K_3})_{11} = \sum_{\substack{e_k \text{ incident} \\ \text{to } v_1}} x_k = x_1 + x_2,$$

since the edges e_1 and e_2 are the ones incident to v_1, and their weights are x_1 and x_2. Similarly, one verifies that

$$(\mathbf{L}_{K_3})_{22} = x_1 + x_3, \qquad (\mathbf{L}_{K_3})_{33} = x_2 + x_3.$$

The off-diagonal entries here are negated edge weights: we have

$$(\mathbf{L}_{K_3})_{12} = (\mathbf{L}_{K_3})_{21} = -x_1$$

since $e_1 = \{v_1, v_2\}$ is the edge incident to both v_1 and v_2, its corresponding weight is x_1, whence its negation is $-x_1$. We also have

$$(\mathbf{L}_{K_3})_{13} = (\mathbf{L}_{K_3})_{31} = -x_2, \qquad (\mathbf{L}_{K_3})_{23} = (\mathbf{L}_{K_3})_{32} = -x_3.$$

Together, we assemble the weighted Laplacian of the connected triangle graph K_3 as

$$\mathbf{L}_{K_3} = \begin{pmatrix} x_1 + x_3 & -x_1 & -x_2 \\ -x_1 & x_1 + x_3 & -x_3 \\ -x_2 & -x_3 & x_2 + x_3 \end{pmatrix}.$$

Perhaps writing it out in this way has immediately revealed some additional structure: it appears that each row (and each column) of $\mathbf{L}_{K_3}$ *sums to zero*. This is not an accident:

Theorem 4.0.16 (Row-Sum Property of the Laplacian)

For any weighted graph $(G, \mathbf{x})$, each row and each column of $\mathbf{L}_G$ sums to zero — in particular, we have

$$\mathbf{1}^\top \mathbf{L}_G = \mathbf{L}_G \mathbf{1} = \mathbf{0}, \qquad \mathbf{1} = \begin{pmatrix} 1 \\ 1 \\ \vdots \\ 1 \end{pmatrix} \in \mathbb{R}^n \qquad \mathbf{0} = \begin{pmatrix} 0 \\ 0 \\ \vdots \\ 0 \end{pmatrix} \in \mathbb{R}^n.$$

Proof:

Fix a row i. The diagonal entry

$$(\mathbf{L}_G)_{ii} = \sum_{\substack{e_k \text{ incident} \\ \text{to } v_i}} x_k$$

is the sum of the weights of all edges incident to the vertex v_i. The off-diagonal entries in row i are the *negated* weights of those *same* edges — specifically, for each edge $e_k = \{v_i, v_j\}$ incident to v_i, the entry $(\mathbf{L}_G)_{ij}$ contributes $-x_k$ to the row sum. All other off-diagonal entries in row i are zero (corresponding to non-adjacent vertices). If we sum over all of these, we get

$$\sum_{j \neq i} (\mathbf{L}_G)_{ij} = \sum_{\substack{e_k \text{ incident} \\ \text{to } v_i}} (-x_k).$$

Summing across row i:

$$\sum_{j=1}^{n} (\mathbf{L}_G)_{ij} = (\mathbf{L}_G)_{ii} + \sum_{j \neq i} (\mathbf{L}_G)_{ij} = \sum_{\substack{e_k \text{ incident} \\ \text{to } v_i}} x_k + \sum_{\substack{e_k \text{ incident} \\ \text{to } v_i}} (-x_k) = 0.$$

Finally, by the symmetry of $\mathbf{L}_G$, we conclude that the same argument applies to the column-sums: in particular, we conclude $\mathbf{1}^\top \mathbf{L}_G = \mathbf{L}_G \mathbf{1} = \mathbf{0}$.

$\|$

On a weighted graph $(G, \mathbf{x})$, the utility of the weighted Laplacian $\mathbf{L}_G$ is not merely as a storage device; indeed, $\mathbf{L}_G \in \mathbb{R}^{n \times n}$ is a square $(n \times n)$-matrix, and as such, it admits a natural interpretation as a **linear operator** acting on n-dimensional vectors. Indeed, this is the essential distinction between

$$\sum_{j=1}^{n} (\mathbf{L}_G)_{ij} = 0, \qquad i = 1, \dots, n,$$

which reads as "a constraint on the entries $(\mathbf{L}_G)_{ij}$," and

$$\mathbf{L}_G \mathbf{1} = \mathbf{0} \in \mathbb{R}^n.$$

which reads as "a constraint on the *null space* of $\mathbf{L}_G$." The first statement is bookkeeping; the second is *geometry*.

To recapitulate the running metaphor, we're imagining an electrician comes to the foreman with a blueprint $G = (V, E)$ wiring n junction boxes $V = \{v_1, \dots, v_n\}$ by m cables $E = \{e_1, \dots, e_m\}$.

Perhaps we imagine that the vertices $v_i \in V$ were given to the electrician, and they chose the edges $e_k \in E$ to form something of an electrical circuit. We imagine that the foreman's job is to assess the quality of the electrician's schematic; as such, we imagine that to each edge $e_k \in E$, the foreman *assigns* a weight $x_k \in \mathbf{x} \in \mathbb{R}^m$. Since each edge $e_k = \{v_i, v_j\}$ for some vertices v_i, v_j in V, we might imagine the weights x_k relay "how much it costs to lay this particular wire e_k between these particular vertices v_i and v_j."

And at this point, perhaps it is only natural the foreman analogously assigns, to each vertex $v_i \in V$, a number $f_i \in \mathbf{f} \in \mathbb{R}^n$. What should each $f_i \in \mathbf{f}$ associated with $v_i \in V$ represent? The edge weights $x_k \in \mathbf{x}$ were costs associated with the edges $e_k \in E$ — properties of *the cables themselves* that are fixed the second the foreman inspects the blueprint.

The vertex assignments f_i are something different: they *certainly* vary depending on what blueprint the foreman is handed. Indeed, if the vertices are "buildings in a neighborhood" and the edges are "power lines to wire it up," then the vertex assignments f_i are the **voltages** (perhaps "batteries", or substations) that the foreman *chooses and applies* at each building in order to *test the network*. The blueprint is fixed — the electrician has already drawn it, and the foreman has already assessed the cable costs. Now, the foreman is wondering, "If I *hook this network up* — assign each building a voltage and turn on the lights — how does the network *flow*?"

This is the question answered by the weighted Laplacian. Indeed, when the foreman assigns the voltages

$$\mathbf{f} = \begin{pmatrix} f_1 \\ f_2 \\ \vdots \\ f_n \end{pmatrix} \in \mathbb{R}^n$$

to the junction boxes $V = \{v_1, v_2, \dots, v_n\}$ and "turns on the network," we ask: how much **current** flows through each cable $e_k = \{v_i, v_j\}$?

The answer naturally depends on three constraints (two obvious, and one that we take on faith from the experimental physicists). Let us reason through these constraints. Consider a single cable $e_k = \{v_i, v_j\}$, with conductance $x_k > 0$ and voltages f_i, f_j set by the foreman at v_i, v_j.

- **(Current flows from high to low voltage)** If $f_i > f_j$, current flows from v_i to v_j through the cable e_k; if $f_j > f_i$, it flows the other way. If $f_i = f_j$, no current flows at all — this is "equilibrium."

- **(More push means more flow)** The amount of current flowing from v_i to v_j through e_k depends on the *voltage difference* $f_i - f_j$. In particular, we observe that $f_i - f_j > 0 \iff f_i > f_j$, corresponding to the first scenario above where the current is flowing from v_i to v_j. "From the perspective of v_i, a positive $f_i - f_j > 0$ means I need to push more current through the wire to get back to equilibrium." (We take for granted that this relationship is *linear* — this is, of course, well-known and supported extensively by experimental testing.)

- **(Better cables carry more current)** The conductance x_k of the cable determines how much current flows *per unit of voltage difference*. A cable with conductance $x_k = 10$ carries ten times the current of a cable with conductance $x_k = 1$ under the same voltage difference $f_i - f_j$.

Together, these three constraints amount to **Ohm's law**: the current flowing through cable $e_k = \{v_i, v_j\}$ from v_i to v_j is

$$I_k = x_k \cdot (f_i - f_j).$$

If the foreman is standing at junction box v_i, watching current flow in and out, then each cable $e_k = \{v_i, v_j\}$ incident to v_i carries current $I_k = x_k(f_i - f_j)$ *away from* the box v_i (positive means flowing *out*; negative means flowing *in*). The **total current flowing out of** v_i is nothing more than the sum over all the independent contributions from cables touching v_i:

$$(\text{net current out of } v_i) = \sum_{\substack{e_k = \{v_i, v_k\} \\ \text{incident to } v_i}} x_k(f_i - f_j).$$

Let us expand this sum. Splitting the difference, we have

$$\sum_{\substack{e_k = \{v_i, v_j\} \\ \text{incident to } v_i}} x_k(f_i - f_j) = \quad f_i \sum_{\substack{e_k \\ \text{incident to } v_i}} x_k \quad - \quad \sum_{\substack{e_k = \{v_i, v_j\} \\ \text{incident to } v_i}} x_k\, f_j,$$

which splits the sum into two pieces with distinct roles. The first piece is "the voltage f_i at my fixed vertex v_i, multiplied by the total conductance of every cable touching v_i" — this is how much current would flow out of v_i if every neighboring building were grounded at zero volts. The reader recognizes the sum $\sum_{e_k \text{ incident to } v_i} x_k$ as precisely the diagonal entry $(\mathbf{L}_G)_{ii}$ of the weighted Laplacian.

The second piece subtracts, for each neighbor v_j of v_i, the product $x_k f_j$: "the conductance of the cable to v_j, times the voltage at v_j." This is the current that v_j's voltage *pushes back* through the cable toward v_i — it reduces the net outflow because the neighbor is not, in general, sitting at zero. We recognize each factor $-x_k$ as the off-diagonal entry $(\mathbf{L}_G)_{ij}$, and the voltage f_j as the j-th component of $\mathbf{f}$.

So, the first term is $(L_G)_{ii} \cdot f_i$, and the second term is $\sum_{j \neq i} \big(-(L_G)_{ij} \big) \cdot f_j = -\sum_{j \neq i} (L_G)_{ij} f_j$. The net current out of v_i can be written as

$$(\text{net current out of } v_i) = (\mathbf{L}_G)_{ii}\, f_i \; + \; \sum_{j \neq i} (\mathbf{L}_G)_{ij}\, f_j \; = \; \sum_{j=1}^{n} (\mathbf{L}_G)_{ij}\, f_j.$$

But this last expression is nothing more than the i-th entry of the matrix-vector product $\mathbf{L}_G\mathbf{f}$. That is:

$$(\text{net current out of } v_i) = \sum_{j=1}^{n} (\mathbf{L}_G)_{ij}\, f_j = (\mathbf{L}_G \mathbf{f})_i.$$

We have arrived at a clean interpretation of the weighted Laplacian $\mathbf{L}_G \in \mathbb{R}^{n \times n}$: it is the **linear operator** acting on n-dimensional vector of voltages $\mathbf{f} \in \mathbb{R}^n$ set on junction boxes $V = \{v_1, \ldots, v_n\}$, and outputting an n-dimensional vector $\mathbf{L}_G\mathbf{f} \in \mathbb{R}^n$ of *net current flows* with respect to the foreman's weighted graph, $(G, \mathbf{x})$ with $G = (V, E)$.

And now, the row-sum property takes on a *physical* meaning. When we proved $\mathbf{L}_G\mathbf{1} = \mathbf{0}$, what we showed was that the vector $\mathbf{1} = (1, 1, \ldots, 1)^\top \in \mathbb{R}^n$ lies in the **null space** $\mathcal{N}(\mathbf{L}_G)$ of $\mathbf{L}_G$. In the foreman's language, "if every building is set to the same voltage — $f_i \equiv 1$ for all i — then no current flows *anywhere*. There is no voltage difference $f_i - f_j$ across any cable, so Ohm's law gives $I_k = x_k(1 - 1) = 0$ for every cable e_k. The network is in *perfect equilibrium*."

This is not a consequence of any particular graph; it is a consequence of the *physics*. *Any* network, no matter how it is wired, sits in equilibrium when ever node v_i has the same voltage f_i.

More generally, the null space

$$\mathcal{N}(\mathbf{L}_G) = \{\mathbf{f} \in \mathbb{R}^n : \; \mathbf{L}_G\mathbf{f} = \mathbf{0}\}$$

consists of *all* voltage assignments that produce zero net current at every vertex in V. For a *connected* graph, these are precisely the constant vectors

$$\mathbf{f} = c \cdot \mathbf{1}, \qquad c \in \mathbb{R};$$

interpret this to mean "the only way to have zero current *everywhere* is to assign the same voltage to *every building*." (If the graph were *disconnected*, we could assign different constants to different connected components, and the null space would be larger. This is mostly why we restrict attention to connected graphs.)

In the language of the monomial basis $\mathcal{M} = \{1, x, x^2, ...\}$, we observe that connected graphs have null space $\mathcal{N}(\mathbf{L}_G) = \{c \cdot \mathbf{1} : \ c \in \mathbb{R}\}$ — it requires *exactly one basis element*, $1 \in \mathcal{M}$, to *completely pin down* the null space $\mathcal{N}(\mathbf{L}_G)$ of the weighted Laplacian on a connected graph G — in particular, we say that the **dimension** of the null space of $\mathbf{L}_G$ is 1, and write

$$\dim(\mathcal{N}(\mathbf{L}_G)) = 1.$$

So: the weighted Laplacian $\mathbf{L}_G$ of a weighted connected graph is a linear operator on $\mathbb{R}^n$, acting on voltages $\mathbf{f} \in \mathbb{R}^n$ and sending them to vectors of net currents $\mathbf{L}_G\mathbf{f} \in \mathbb{R}^n$. Furthermore, $\mathbf{L}_G$ has a one-dimensional null space $\dim(\mathcal{N}(\mathbf{L}_G)) = 1 > 0$, and the null space $\mathcal{N}(\mathbf{L}_G)$ is **spanned by** the vector $\mathbf{1} \in \mathbb{R}^n$.

The foreman now asks a natural question: "I know that constant voltages produce no current. But what about *non-constant* voltages? If I set different buildings to different voltages, *how strongly* does the network respond?"

It turns out that, at its core, the foreman's question is one about **determinants**.

An Aside on Determinants

To answer this, let us step back from graphs for a moment and consider any linear operator $\mathbf{A} \in \mathbb{R}^{n\times n}$ acting on vectors $\mathbf{v} \in \mathbb{R}^n$. We ask: what does $\mathbf{A}$ *do* to the space $\mathbb{R}^n$? It stretches some directions, compresses others, perhaps flips some, and — if $\mathcal{N}(\mathbf{A}) \neq \{\mathbf{0}\}$ — it *collapses* certain directions entirely, sending them to $\mathbf{0}$.

The foreman's intuition is geometric. Imagine the unit cube in $\mathbb{R}^n$: the set of all points $\mathbf{v} = c_1\mathbf{e}_1 + c_2\mathbf{e}_2 + \cdots + c_n\mathbf{e}_n$ with $0 \leq c_i \leq 1$. This cube has n-dimensional volume equal to 1. Now apply $\mathbf{A}$: each basis vector $\mathbf{e}_i$ maps to the i-th column of $\mathbf{A}$, and the unit cube deforms into a **parallelepiped** — a slanted, stretched, possibly flattened version of itself. The n-dimensional volume of this parallelepiped measures *how much* $\mathbf{A}$ *inflates or deflates n-dimensional content.*

This "volume scaling factor" is precisely the **determinant** of $\mathbf{A}$.

Definition 4.0.17 (Determinant (Geometric))

Let $\mathbf{A} \in \mathbb{R}^{n\times n}$ be a square matrix with columns $\mathbf{a}_1, \mathbf{a}_2, \ldots, \mathbf{a}_n \in \mathbb{R}^n$. The **determinant** $\det(\mathbf{A})$ is the *signed* n-dimensional volume of the parallelepiped

$$\mathcal{P}(\mathbf{A}) := \left\{c_1\mathbf{a}_1 + c_2\mathbf{a}_2 + \cdots + c_n\mathbf{a}_n \ : \ 0 \leq c_i \leq 1\right\}.$$

The sign is positive if $\mathbf{A}$ preserves orientation (the columns wind in the same direction as the standard basis) and negative if $\mathbf{A}$ reverses it.

The reader may wonder what "signed volume" means. In two dimensions, it is easy to visualize: two vectors $\mathbf{a}_1, \mathbf{a}_2 \in \mathbb{R}^2$ span a parallelogram. If $\mathbf{a}_2$ lies counterclockwise from $\mathbf{a}_1$, the signed area

is positive; if clockwise, negative. In higher dimensions, the sign generalizes through orientation — a concept that, for our purposes, is best treated as "a bookkeeping device that tracks whether the columns of $\mathbf{A}$ can be continuously deformed to the standard basis without passing through a degenerate configuration."

What *matters* for us is the following: the magnitude $|\det(\mathbf{A})|$ measures how much $\mathbf{A}$ scales volume, and the condition $\det(\mathbf{A}) = 0$ means the parallelepiped has been *flattened* — $\mathbf{A}$ has collapsed n-dimensional content into something of strictly lower dimension. In other words:

Theorem 4.0.18 (Determinant and Invertibility)

A matrix $\mathbf{A} \in \mathbb{R}^{n\times n}$ satisfies $\det(\mathbf{A}) = 0$ if and only if $\mathcal{N}(\mathbf{A}) \neq \{\mathbf{0}\}$ — that is, if and only if $\mathbf{A}$ is *not* invertible.

Proof: If $\mathcal{N}(\mathbf{A}) \neq \{\mathbf{0}\}$, then there exists some nonzero $\mathbf{v} \in \mathbb{R}^n$ with $\mathbf{Av} = \mathbf{0}$. This means $\mathbf{v}$ lies in the span of the columns of $\mathbf{A}$ with a nontrivial linear dependence — the columns are linearly dependent, the parallelepiped they span is degenerate (contained in a hyperplane), and its n-dimensional volume is zero.

Conversely, if the columns of $\mathbf{A}$ are linearly independent, they span a full n-dimensional parallelepiped with nonzero volume, and $\mathbf{A}$ maps $\mathbb{R}^n$ onto $\mathbb{R}^n$ bijectively — it is invertible, with $\mathcal{N}(\mathbf{A}) = \{\mathbf{0}\}$.

$\|$

Now let us return to the foreman's Laplacian $\mathbf{L}_G$. We showed that $\dim(\mathcal{N}(\mathbf{L}_G)) = 1$ for a connected graph — the null space is spanned by $\mathbf{1}$. By the theorem above:

$$\det(\mathbf{L}_G) = 0.$$

Geometrically, the Laplacian takes the unit cube in $\mathbb{R}^n$ (the space of all possible voltage assignments) and maps it to a parallelepiped of *zero n-dimensional volume* in the space of net currents. The image is flat — it lives in an $(n-1)$-dimensional hyperplane, because the "constant voltage" direction gets crushed to zero, and only the $n-1$ "non-constant" directions survive.

This is the foreman's disappointment, and also their opportunity. The full determinant $\det(\mathbf{L}_G) = 0$ tells the foreman nothing — it is zero for *every* connected graph, regardless of the wiring. But the *$(n-1)$-dimensional* volume of the image — the volume of the parallelepiped $\mathbf{L}_G$ produces in the $(n-1)$-dimensional subspace orthogonal to $\mathbf{1}$ — is *not* zero. It depends on the graph, it depends on the weights, and (as we will see) it counts spanning trees.

To extract this $(n-1)$-dimensional volume, we simply remove the direction that gets crushed. We delete one row and one column from $\mathbf{L}_G$ — it does not matter which, since the null space is symmetric across all vertices — and take the determinant of the remaining $(n-1)\times(n-1)$ matrix. This is the **cofactor**, and its determinant measures the $(n-1)$-dimensional volume of the Laplacian's image.

But to compute this determinant, we need a *formula*. The geometric definition — "the volume of the parallelepiped" — is fine for intuition, but the foreman needs an algebraic expression in terms of the matrix entries $(\mathbf{L}_G)_{ij}$. This is the **Leibniz formula**, and we derive it now.

Consider: what properties *must* any reasonable notion of "n-dimensional volume" satisfy? We claim there are exactly three, and they uniquely determine the determinant.

Definition 4.0.19 (Determinant (Axiomatic))

The **determinant** is the unique function $D\colon \mathbb{R}^{n\times n} \to \mathbb{R}$ satisfying:

(i) **Normalization.** $D(\mathbf{I}) = 1$. "The unit cube has volume one."

(ii) **Degeneracy.** If two columns of $\mathbf{A}$ are equal, $D(\mathbf{A}) = 0$. "A parallelepiped with two identical edges is flat."

(iii) **Multilinearity.** D is linear in each column separately: if we fix all columns except the j-th and write $\mathbf{a}_j = \mathbf{v} + c\,\mathbf{w}$, then

$$D(\dots, \mathbf{v} + c\,\mathbf{w}, \dots) = D(\dots, \mathbf{v}, \dots) + c \cdot D(\dots, \mathbf{w}, \dots).$$

"Scaling one edge of the parallelepiped scales the volume proportionally."

Again, we assert these properties intuitively[a], but we claim that they really are irresistable if one simply places themselves in the mind of the foreman. Indeed, if we accept them, we recover some familiar properties:

Theorem 4.0.20 (Alternating Property)

If D satisfies properties (1)–(3), then swapping any two columns of $\mathbf{A}$ *negates* D:

$$D(\dots, \mathbf{a}_i, \dots, \mathbf{a}_j, \dots) = -D(\dots, \mathbf{a}_j, \dots, \mathbf{a}_i, \dots).$$

Proof: By degeneracy (property 2), $D(\dots, \mathbf{a}_i + \mathbf{a}_j, \dots, \mathbf{a}_i + \mathbf{a}_j, \dots) = 0$. Expanding by multilinearity (property 3) in both columns:

$$\begin{aligned} 0 = {} & D(\dots, \mathbf{a}_i, \dots, \mathbf{a}_i, \dots) + D(\dots, \mathbf{a}_i, \dots, \mathbf{a}_j, \dots) \\ & + D(\dots, \mathbf{a}_j, \dots, \mathbf{a}_i, \dots) + D(\dots, \mathbf{a}_j, \dots, \mathbf{a}_j, \dots). \end{aligned}$$

The first and fourth terms vanish by degeneracy (repeated columns). What remains is

$$D(\dots, \mathbf{a}_i, \dots, \mathbf{a}_j, \dots) + D(\dots, \mathbf{a}_j, \dots, \mathbf{a}_i, \dots) = 0,$$

which is precisely the alternating property.

‖

So volume is *signed*: swapping two edges of the parallelepiped flips the sign. This is why we speak of *signed* volume — the three axioms *force* it upon us.

Now, any column $\mathbf{a}_j$ of $\mathbf{A}$ can be expanded in the standard basis: $\mathbf{a}_j = \sum_{i=1}^{n} a_{ij}\,\mathbf{e}_i$, where a_{ij} is the (i,j)-entry of $\mathbf{A}$. By multilinearity, we may expand $D(\mathbf{a}_1, \dots, \mathbf{a}_n)$ one column at a time, pulling out each scalar a_{ij}:

$$D(\mathbf{A}) = D\left(\sum_{i_1} a_{i_1,1}\mathbf{e}_{i_1},\ \sum_{i_2} a_{i_2,2}\mathbf{e}_{i_2},\ \dots,\ \sum_{i_n} a_{i_n,n}\mathbf{e}_{i_n} \right)$$

$$= \sum_{i_1,i_2,\dots,i_n=1}^{n} a_{i_1,1}\, a_{i_2,2} \cdots a_{i_n,n}\, D(\mathbf{e}_{i_1}, \mathbf{e}_{i_2}, \dots, \mathbf{e}_{i_n}).$$

This is a sum over all n^n possible index tuples $(i_1, \dots, i_n)$. But most terms vanish: by the degeneracy property, $D(\mathbf{e}_{i_1}, \dots, \mathbf{e}_{i_n}) = 0$ whenever any two indices coincide ($i_j = i_k$ for $j \neq k$). The only surviving terms are those where $(i_1, \dots, i_n)$ is a **permutation** of $\{1, 2, \dots, n\}$ — that is, where each standard basis vector appears exactly once.

A **permutation** of $\{1, \dots, n\}$ is a bijection $\sigma \colon \{1, \dots, n\} \to \{1, \dots, n\}$. There are exactly $n!$ such bijections, and we denote the set of all of them by S_n.

For any permutation $\sigma \in S_n$, the matrix $\mathbf{E}_\sigma := [\mathbf{e}_{\sigma(1)}, \mathbf{e}_{\sigma(2)}, \dots, \mathbf{e}_{\sigma(n)}]$ is obtained from the identity by rearranging its columns according to σ. By the alternating property, each column swap contributes a factor of (-1), and the total sign accumulated in transforming $\mathbf{E}_\sigma$ back to $\mathbf{I}$ is denoted $\operatorname{sgn}(\sigma) \in \{+1, -1\}$. By normalization, $D(\mathbf{I}) = 1$, so $D(\mathbf{E}_\sigma) = \operatorname{sgn}(\sigma)$.

(The **sign** of a permutation is $+1$ if σ can be decomposed into an even number of transpositions — swaps of two elements — and -1 if odd. One can show that this parity is well-defined: the number of transpositions varies, but the parity does not.)

Substituting $i_j = \sigma(j)$ into our expansion, the sum over n^n tuples collapses to a sum over $n!$ permutations:

Theorem 4.0.21 (Leibniz Formula for the Determinant)

Let $\mathbf{A} = \Big((A_{i,j})\Big) \in \mathbb{R}^{n \times n}$ be a square matrix. Denote by S_n the set of all $n!$ permutations $\sigma \colon \{1, \dots, n\} \to \{1, \dots, n\}$, and by $\operatorname{sgn}(\sigma) \in \{+1, -1\}$ the **sign** of σ — equal to $+1$ if σ decomposes into an even number of transpositions, and -1 if odd.

Then, the determinant of $\mathbf{A}$ is defined to be

$$\det(\mathbf{A}) := \sum_{\sigma \in S_n} \operatorname{sgn}(\sigma) \prod_{j=1}^{n} A_{\sigma(j),j},$$

which is to be read as

$$\det(\mathbf{A}) \quad := \quad \overbrace{\sum_{\sigma \in S_n}}^{\substack{\text{sum over all } n! \\ \text{permutations } \sigma \in S_n}} \quad \underbrace{\operatorname{sgn}(\sigma)}_{\substack{+1 \text{ if } \sigma \text{ is even,} \\ -1 \text{ if } \sigma \text{ is odd}}} \quad \overbrace{\prod_{j=1}^{n} A_{\sigma(j),j}}^{\substack{\text{from column } j, \\ \text{select row } \sigma(j); \\ \text{each row used exactly once}}}$$

We go down the line of permutations $\sigma \in S_n$. Each individual term $\operatorname{sgn}(\sigma) \prod_{j=1}^{n} A_{\sigma(j),j}$ associated with the permutation $\sigma \in S_n$ selects one entry from each column of $\mathbf{A}$ — specifically, from row $\sigma(j)$ of column j — multiplies them together, and weights the product by the sign of the permutation.

The determinant is the signed sum over all $n!$ ways to select exactly one element from each row and each column of $\mathbf{A}$, multiply the n selected elements together, and weight by the sign of the permutation $\sigma \in S_n$ dictating the selection.

For a 2×2 matrix, there are $2! = 2$ permutations, and the formula gives the familiar

$$\det \begin{pmatrix} a & b \\ c & d \end{pmatrix} = ad - bc.$$

For a 3×3 matrix, there are $3! = 6$ permutations, giving the "rule of Sarrus" expansion with six terms. For an $n \times n$ matrix, the $n!$ terms grow rapidly — but the formula is *exact*, and for symbolic computation (where the entries are formal variables rather than floating-point numbers), it is the foundation upon which everything else rests.

Remark.

One really must find their own flavor of determinant characterization that works for them. Leibniz's identity is often reprimanded for lacking clarity, but we find it perfectly clear, viewed through the lens of chess.

Indeed, imagine an $n \times n$ grid of numbers — the entries of $\mathbf{A}$. Perhaps we imagine $n = 8$, so that $\mathbf{A}$ is *quite literally* "shaped like a chess board" — but regardless, we're tasked with placing n non-attacking rooks on the grid: one per row, one per column, with no two rooks sharing the same row or column. (If one doesn't know how the rooks move in a game of chess, perhaps this analogy isn't for you, but perhaps it is sufficient to understand that they "move up and down, left and right, but not diagonally.")

When we imagine the ways we can place n non-attacking rooks on an $n \times n$ board, perhaps we'd start by appealing to the rooks' impotence along the diagonal and, letting 1s denote where we place the rooks, set up the board

$$\begin{pmatrix} 1 & 0 & \cdots & 0 \\ 0 & 1 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & 1 \end{pmatrix}.$$

This is certainly the most obvious configuration to *start with*, but once we've identified that it works, the procedure for generating all other valid placements is quite literally to *permute* the rows (or columns) of this initial arrangement. Each of the $n!$ permutations $\sigma \in S_n$ gives a valid placement: the rook in column j sits in row $\sigma(j)$. There are no other valid placements — every non-attacking rook configuration is a permutation of the diagonal.

For example, the $3! = 6$ **permutation matrices** in $\mathbb{R}^{3\times 3}$ — those which are nothing more than "rearrangements of the 3-dimensional identity, *including itself*" — are

$$\begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}, \quad \begin{pmatrix} 1 & 0 & 0 \\ 0 & 0 & 1 \\ 0 & 1 & 0 \end{pmatrix},$$

$$\begin{pmatrix} 0 & 1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{pmatrix}, \quad \begin{pmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ 1 & 0 & 0 \end{pmatrix},$$

$$\begin{pmatrix} 0 & 0 & 1 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \end{pmatrix}, \quad \begin{pmatrix} 0 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 0 & 0 \end{pmatrix}.$$

Now, in ordinary chess, the n rooks are identical — a rook is a rook, and we care only about *where* they sit, not *what's underneath them*. This is the setting most naturally associated with permutations of the identity, as illustrated above.

But in general, the determinant *wants* to play a different, *more general* game than "ordinary chess." Indeed, imagine instead that every square on the board has a number written on it — the

entry A_{ij} of $\mathbf{A}$. When we place n rooks in a valid configuration σ, each rook lands on a number: the rook in column j lands on $A_{\sigma(j),j}$. We multiply these n numbers together to get a **score** for that configuration:

$$\text{score}(\sigma) = \prod_{j=1}^{n} A_{\sigma(j),j}.$$

This is chess on a board where the squares have different *values*. The diagonal of the identity matrix is a board of all 1s, and every configuration scores 1. A diagonal matrix is a board where only the main diagonal has nonzero values, so only the identity configuration scores anything at all. A general matrix is a board where every square might have a different number, and different rook placements produce different scores.

But the determinant does not simply *sum* the scores — it plays them *against each other*. Each of the $n!$ configurations is assigned to one of two teams based on the sign of its permutation: the even permutations ($\text{sgn}(\sigma) = +1$) play for one side, the odd permutations ($\text{sgn}(\sigma) = -1$) play for the other, and determinant is the *final score* of the game:

$$\det(\mathbf{A}) \quad = \sum_{\substack{\text{even} \\ \text{permutations } \sigma}} \text{score}(\sigma) \quad - \sum_{\substack{\text{odd} \\ \text{permutations } \sigma}} \text{score}(\sigma).$$

When the determinant is large, one team is dominant — the winning configurations are consistently landing on larger numbers than the losing configurations. When the determinant is zero, the two teams are perfectly balanced — every gain on one side is exactly offset by a gain on the other. And *this* is the geometric content: a zero determinant means the columns of $\mathbf{A}$ are linearly dependent, the parallelepiped is flat, and the perfect balance of the two teams is the algebraic manifestation of that flatness.

The reader may find it useful to think of the 2×2 case. There are $2! = 2$ configurations: the identity $\sigma = \text{id}$ places rooks at $(1,1)$ and $(2,2)$, scoring $a_{11}a_{22}$; the swap $\sigma = (1\ 2)$ places rooks at $(2,1)$ and $(1,2)$, scoring $a_{21}a_{12}$. The identity is even $(+1)$, the swap is odd (-1), and:

$$\det \begin{pmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{pmatrix} = a_{11}a_{22} - a_{21}a_{12}.$$

Two rook placements, one per team, and the familiar formula is just "the diagonal team minus the anti-diagonal team."

---

[a] The interested reader is encouraged to see, e.g., Fortney's explanation.

Back to the Laplacian

Let us now return to the foreman's Laplacian. We have:

- A matrix $\mathbf{L}_G \in \mathbb{R}^{n \times n}$ built from the graph's edge weights.
- A one-dimensional null space $\mathcal{N}(\mathbf{L}_G) = \text{span}(\mathbf{1})$, so $\det(\mathbf{L}_G) = 0$.
- The cofactor $\mathbf{L}_G^{(i)}$: the $(n-1) \times (n-1)$ matrix obtained by deleting row i and column i.

We verified, for the triangle K_3, that $\det(\mathbf{L}_{K_3}^{(i)}) = x_1x_2 + x_1x_3 + x_2x_3 = K_{K_3}(\mathbf{x})$ — the Kirchhoff polynomial, summing over all spanning trees. We verified that the choice of i does not matter:

all three cofactors produce the same polynomial. This is the content of the matrix-tree theorem, which we are now equipped to state and prove.

Theorem 4.0.22 (Matrix-Tree (Kirchoff, 1847))

Let $(G, \mathbf{x})$ be a connected weighted graph on n vertices, with weighted Laplacian $\mathbf{L}_G$. Then for *any* choice of vertex $v_i \in V$, the Kirchoff polynomial

$$K_G(x_1, \dots, x_m) = \det\left(\mathbf{L}_G^{(-i)}\right),$$

where

$$\mathbf{L}_G^{(i)} = \left(\begin{array}{ccc|c|ccc} \ell_{1,1} & \cdots & \ell_{1,i-1} & i\text{th} & \ell_{1,i+1} & \cdots & \ell_{1,n} \\ \vdots & \ddots & \vdots & & \vdots & \ddots & \vdots \\ \ell_{i-1,1} & \cdots & \ell_{i-1,i-1} & & \ell_{i-1,i+1} & \cdots & \ell_{i-1,n} \\ \hline i\text{th} & & & & & & \\ \hline \ell_{i+1,1} & \cdots & \ell_{i+1,i-1} & & \ell_{i+1,i+1} & \cdots & \ell_{i+1,n} \\ \vdots & \ddots & \vdots & & \vdots & \ddots & \vdots \\ \ell_{n,1} & \cdots & \ell_{n,i-1} & & \ell_{n,i+1} & \cdots & \ell_{n,n} \end{array}\right) \in \mathbb{R}^{(n-1)\times(n-1)}$$

is the $(n-1) \times (n-1)$ **cofactor** matrix obtained by deleting row i and column i from $\mathbf{L}_G$.

But before *proving* the matrix-tree theorem, we want to understand *why* it should be true — what mechanism inside the Leibniz determinant formula could possibly produce a *sum over spanning trees*? Indeed, we know that, by definition,

$$K_G(x_1, \dots, x_m) := \sum_{\substack{\text{spanning} \\ \text{trees } T}} \prod_{e_k \in T} x_k,$$

so what we're really saying here is that